

Technische Dokumentation – JPac

1. Einleitung

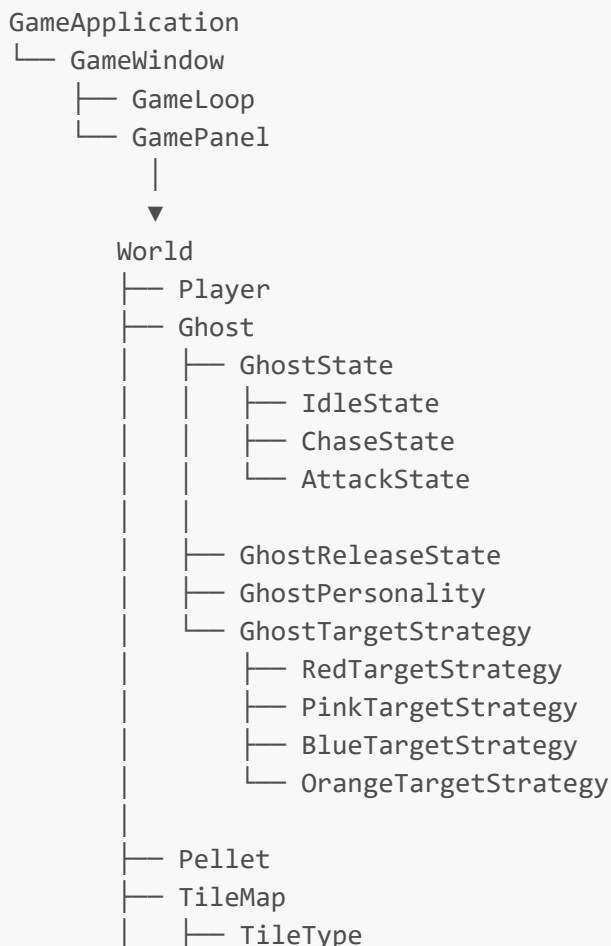
JPac ist ein Pac-Man-inspiriertes Spiel, das im Rahmen einer Semesterarbeit in Java entwickelt wurde. Der Spieler navigiert durch ein kachelbasiertes Labyrinth, sammelt Pellets ein und weicht vier Geistern aus, die unterschiedliche Verfolgungsstrategien verfolgen.

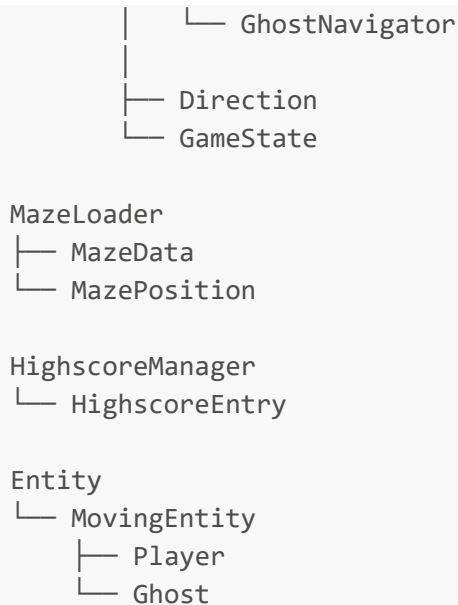
Der technische Fokus liegt auf einer klaren Trennung von Spiellogik, KI-Verhalten und Darstellung. Das Projekt demonstriert den praxisnahen Einsatz objektorientierter Entwurfsmuster, einen datengetriebenen Ansatz für Spielinhalte sowie eine robuste Zustandsverwaltung über den gesamten Spielablauf.

Der Umfang umfasst eine vollständige Spielanwendung mit Hauptmenü, Namenseingabe, Labyrinthauswahl, laufendem Spiel, Endbildschirmen und einem persistenten Top-10-Highscore-System. Die Spiellogik ist durch 72 automatisierte JUnit-Tests abgedeckt.

2. Systemübersicht

Die Anwendung ist in fünf Pakete gegliedert. Die Hauptverantwortlichkeiten sind klar voneinander getrennt: Spiellogik liegt im `world`-Paket, Rendering im `ui`-Paket, die Game Loop im `engine`-Paket, Labyrinthverarbeitung im `maze`-Paket und Datenhaltung im `persistence`-Paket.





Die Kommunikation folgt einer klaren Richtung: `GameLoop` ruft pro Frame `world.update(deltaTime)` auf und löst danach `panel.repaint()` aus. `GamePanel` liest bei jedem Zeichenvorgang den aktuellen Zustand aus `World` – es schreibt nie zurück. Benutzereingaben werden in `GamePanel` empfangen und als Methodenaufrufe an `World` weitergeleitet (z. B. `world.confirmPlayerName(...)`, `world.startMaze(...)`), die ihrerseits den Spielzustand aktualisieren.

3. Paketstruktur

`ffhs.jpac`

Enthält ausschliesslich die Einstiegsklasse `GameApplication` mit der `main`-Methode. Sie erzeugt ein `GameWindow` und macht es sichtbar.

`ffhs.jpac.engine`

Zuständig für den Spieltakt.

Klasse	Aufgabe
<code>GameLoop</code>	Führt die Spielschleife in einem eigenen Thread aus, reguliert auf 60 FPS, übergibt Delta-Time an <code>World</code>

Die Loop berechnet die tatsächlich verstrichene Zeit zwischen zwei Frames (`deltaTime`) in Sekunden und übergibt sie an `world.update(deltaTime)`. Anschliessend wird `panel.repaint()` aufgerufen. Wenn ein Frame schneller als das 60-FPS-Ziel abgeschlossen ist, schläft der Thread die Restzeit.

`ffhs.jpac.ui`

Zuständig für Darstellung und Eingabeverarbeitung. Enthält keine Spiellogik.

Klasse	Aufgabe
<code>GameWindow</code>	Erstellt das <code>JFrame</code> , initialisiert <code>World</code> mit dem Standardlabyrinth, startet <code>GameLoop</code>

Klasse	Aufgabe
<code>GamePanel</code>	Zeichnet alle Spielzustände, verarbeitet Tastatur- und Mauseingaben
<code>TileRenderer</code>	Zeichnet das Kachelgitter ausschnittsweise anhand von Kamerakoordinaten

ffhs.jpac.world

Das Herzstück der Anwendung. Enthält alle Spielobjekte, Zustände, KI-Logik und Kollisionsverarbeitung.

Klasse / Interface	Aufgabe
<code>World</code>	Zentrale Spiellogik, Zustandsverwaltung, Kollisionsprüfung, Highscore-Auslösung
<code>Entity</code>	Abstrakte Basis für alle Spielobjekte (Position, Grösse, Spawn-Reset)
<code>MovingEntity</code>	Erweitert <code>Entity</code> um Geschwindigkeit, Delta-Time-Bewegung, Lane-Korrektur
<code>Player</code>	Spielersteuerung mit Input-Buffering und Kachelzentrierung
<code>Ghost</code>	Geisterlogik: Release-Zustand, KI-Zustand, Persönlichkeitsstrategie
<code>Pellet</code>	Statisches Spielobjekt, speichert Einsammelstatus
<code>TileMap</code>	Kachelgitter, Spawnpunkte, Pfadfindungs-Delegation
<code>GhostNavigator</code>	BFS-basierte Pfadfindung für Geister
<code>GhostTargetStrategy</code>	Interface für austauschbare Zielfindungsstrategien
<code>RedTargetStrategy</code>	Verfolgt direkt die aktuelle Spielerposition
<code>PinkTargetStrategy</code>	Visiert eine Position vor dem Spieler an
<code>BlueTargetStrategy</code>	Patrouilliert mit zeitgesteuerten Angriffsphasen
<code>OrangeTargetStrategy</code>	Wechselt mit Hysterese zwischen Verfolgen und Rückzug
<code>GhostState</code>	Interface für Ghost-Verhaltenszustände
<code>IdleState</code>	Geist bewegt sich ohne Zielverfolgung
<code>ChaseState</code>	Geist verfolgt aktiv den Spieler
<code>AttackState</code>	Identisch mit <code>ChaseState</code> ; aktiviert bei Nahbereich
<code>GameState</code>	Enum der Spielzustände
<code>GhostReleaseState</code>	Enum des Geisterhaus-Austritt-Zyklus
<code>GhostPersonality</code>	Enum der vier Geisterpersönlichkeiten
<code>Direction</code>	Enum mit Richtungsvektoren und <code>opposite()</code> -Methode
<code>TileType</code>	Enum: <code>WALL</code> , <code>FLOOR</code> , <code>GHOST_HOUSE</code>

ffhs.jpac.maze

Zuständig für das Laden und Validieren von Labyrinthdaten.

Klasse	Aufgabe
<code>MazeLoader</code>	Liest JSON-Labyrinthressourcen, führt mehrstufige Validierung durch
<code>MazeData</code>	Datentransferklasse mit <code>id</code> , <code>name</code> und <code>pattern</code>
<code>MazePosition</code>	Unveränderlicher Record für eine Gitterposition (<code>row</code> , <code>col</code>)

`ffhs.jpac.persistence`

Zuständig für die Highscore-Persistenz.

Klasse	Aufgabe
<code>HighscoreManager</code>	Liest und schreibt <code>highscores.json</code> , verwaltet Top-10-Liste
<code>HighscoreEntry</code>	Datensatz mit Name, Punktzahl, Labyrinth-ID und Labyrinthname

4. Verwendete Entwurfsmuster

Strategy Pattern – Ghost-Zielfindung

Wo: Interface `GhostTargetStrategy`, Klassen `RedTargetStrategy`, `PinkTargetStrategy`, `BlueTargetStrategy`, `OrangeTargetStrategy`

Jeder Geist erhält im Konstruktor eine konkrete Strategieimplementierung. Die Methode `targetStrategy.getTarget(ghost, world)` liefert die Zielkachel, die `ghost.chasePlayer(world)` dann zur Pfadfindung übergibt. Der `Ghost` selbst enthält keinerlei Persönlichkeitslogik – er delegiert vollständig an die Strategie.

```

GhostTargetStrategy
├── RedTargetStrategy
├── PinkTargetStrategy
├── BlueTargetStrategy
└── OrangeTargetStrategy

```

Vorteil: Neue Persönlichkeiten lassen sich hinzufügen, ohne `Ghost` zu verändern. Jede Strategie ist isoliert testbar.

State Pattern – Ghost-Verhaltenszustände

Wo: Interface `GhostState`, Klassen `IdleState`, `ChaseState`, `AttackState`; Attribut `currentState` in `Ghost`

Im `Ghost.update()`-Aufruf wird nach Abschluss der Hausaustrittsphase `updateState(world)` aufgerufen, das anhand der Distanz zum Spieler entweder `ChaseState` (Distanz ≥ 20 Pixel) oder `AttackState` (Distanz < 20 Pixel) setzt. Anschliessend delegiert `Ghost` an `currentState.update(this, world, deltaTime)`.

```
GhostState
├── IdleState      → ghost.chooseIdleDirection(); ghost.move()
├── ChaseState     → ghost.chasePlayer(); ghost.move()
└── AttackState    → (extends ChaseState, selbes Verhalten)
```

Vorteil: Das Verhalten pro Zustand ist in einer einzigen, kurzen Klasse gekapselt. `Ghost.update()` enthält keine langen Bedingungsketten.

Separation of Concerns

Wo: Paketgrenzen zwischen `world`, `ui`, `engine`, `maze`, `persistence`

`World` hat keine Abhängigkeit auf `javax.swing`. `GamePanel` enthält keine Spiellogik – es liest Zustände und löst Übergänge per Methodenaufruf aus. `GhostNavigator` ist von `Ghost` getrennt und hat keine Kenntnis von Spielzuständen oder Entitäten. `MazeLoader` arbeitet ausschliesslich auf `MazeData` und `List<String>`, ohne Kopplung an `TileMap`.

Diese Trennung ermöglicht es, die Spiellogik in Unit-Tests vollständig ohne Swing-Komponenten zu testen.

5. Zentrale Klassen

`World`

Aufgabe: Verwaltung der gesamten Spiellogik. `World` kennt alle Entitäten, alle Pellets, die Karte, den aktuellen Spielzustand und die Highscore-Verwaltung.

Wichtige Attribute:

Attribut	Typ	Beschreibung
<code>gameState</code>	<code>GameState</code>	Aktueller Spielzustand
<code>entities</code>	<code>List<Entity></code>	Alle aktiven Spielobjekte
<code>pellets</code>	<code>List<Pellet></code>	Alle Pellets im Labyrinth
<code>map</code>	<code>TileMap</code>	Aktive Karte
<code>score</code>	<code>int</code>	Aktueller Punktestand
<code>playerName</code>	<code>String</code>	Spielername, bleibt bei Neustart erhalten

Wichtige Methoden:

- `update(double deltaTime)` — Hauptschleife: aktualisiert alle Entitäten, prüft Pellet-Einsammeln, prüft Geisterkollision, prüft Gewinnbedingung. Läuft nur bei `GameState.PLAYING`.
- `canMove(Entity, Direction, double)` — Kollisionsprüfung via temporäres Verschieben der Entität
- `isBlockedFor(Entity, int row, int col)` — Bestimmt, ob eine Kachel für eine gegebene Entität blockiert ist; Geisterhausregeln variieren je nach Entitätstyp

- `initializeMaze(TileMap)` — Erstellt Spieler, Geister und Pellets für ein neues Labyrinth
- `restartGame()` — Lädt das aktuelle Labyrinth neu; behält Spielernamen
- `returnToMainMenu()` — Setzt Sitzung zurück, wechselt in `START_MENU`

Beziehungen: Kennt `TileMap`, `Player`, `Ghost`, `Pellet`, `HighscoreManager`. Wird von `GameLoop` (update) und `GamePanel` (read + trigger) verwendet.

Entity / MovingEntity

Aufgabe: Bilden die Vererbungshierarchie aller Spielobjekte.

```

Entity      (x, y, size, spawnX, spawnY)
├── MovingEntity  (+ dx, dy, speed)
│   ├── Player
│   └── Ghost
└── Pellet      (extends Entity, kein MovingEntity)
  
```

`Entity` speichert die Spawn-Position und stellt `reset()` bereit, das zur Ausgangsposition zurückkehrt.

`MovingEntity` implementiert `move(world, deltaTime)` mit automatischer Lane-Korrektur: Bewegt sich eine Entität horizontal, wird ihre Y-Position kontinuierlich auf die nächste Kachelmitte zentriert und umgekehrt. Das verhindert Drift aus den Korridoren.

Player

Aufgabe: Verarbeitung von Spielereingaben mit Eingabepufferung und kachelbasierter Bewegungslogik.

Wichtige Attribute:

Attribut	Beschreibung
<code>currentDirection</code>	Aktiv ausgeführte Richtung
<code>desiredDirection</code>	Gepufferte Wunschrichtung
<code>inputBufferTimer</code>	Verbleibende Gültigkeitsdauer des gepufferten Inputs (250 ms)

Wichtige Methoden:

- `setDesiredDirection(Direction)` — Setzt Wunschrichtung und startet den Puffer-Timer
- `update(World, double)` — Kachel-basierte Bewegungsschleife: prüft an jedem Kachelmittelpunkt, ob ein Richtungswechsel möglich ist, verarbeitet den Eingabepuffer, bewegt sich segmentweise auf den nächsten Mittelpunkt zu

Bewegungslogik im Detail:

Der Spieler bewegt sich ausschliesslich von Kachelmitte zu Kachelmitte. Bei jedem Frame wird geprüft, ob die aktuelle Position nahe genug an einem Mittelpunkt liegt (Toleranz: 2 Pixel). Ist dies der Fall, wird die Kachelmitte exakt gesetzt (`snapToTileCenter`) und die Richtungsentscheidung getroffen. Die

Wunschrichtung wird bis zu 250 ms lang gepuffert – drückt der Spieler eine Richtung kurz vor einer Ecke, wird sie beim nächsten erreichbaren Kachelmittelpunkt angewendet.

Ghost

Aufgabe: Verwaltet den Lebenszyklus eines Geistes vom Warten im Haus bis zur aktiven Verfolgung.

Wichtige Attribute:

Attribut	Beschreibung
<code>releaseState</code>	<code>WAITING_IN_HOUSE</code> , <code>LEAVING_HOUSE</code> , <code>ACTIVE</code>
<code>releaseDelaySeconds</code>	Wartezeit vor dem Verlassen (0 / 5 / 10 / 15 s)
<code>currentState</code>	Aktueller <code>GhostState</code>
<code>targetStrategy</code>	Persönlichkeitsspezifische <code>GhostTargetStrategy</code>
<code>lastDecisionRow/Col</code>	Letzte Kachel, auf der eine Richtungsentscheidung getroffen wurde

Wichtige Methoden:

- `update(World, double)` — Hauptablauf: Wartezeit herunterzählen → Haus verlassen → KI-Zustand aktualisieren → `currentState.update()` delegieren
- `leaveGhostHouse(World, double)` — Navigiert via `GhostNavigator` zum Ausgangs-Tile; setzt `releaseState` auf `ACTIVE` bei Ankunft
- `chasePlayer(World)` — Ruft `targetStrategy.getTarget()` ab, bestimmt via `TileMap.getDirectionTowardTarget()` die nächste Richtung
- `chooseIdleDirection(World)` — Wählt Bewegungsrichtung ohne Zielverfolgung; bevorzugt Geradeausfahrt, verhindert Umkehrung

TileMap

Aufgabe: Repräsentiert das geladene Labyrinth als zweidimensionales Array und stellt alle kartenspezifischen Abfragen bereit.

Wichtige Attribute:

Attribut	Beschreibung
<code>map</code>	<code>TileType[][]</code> – Kacheltypen des Labyrinths
<code>pelletTiles</code>	<code>boolean[][]</code> – Markierung von Pellet-Kacheln
<code>ghostSpawns</code>	<code>List<MazePosition></code> – Startpositionen der Geister
<code>ghostHouseExits</code>	<code>List<MazePosition></code> – Ausgangskacheln des Geisterhauses
<code>playerSpawn</code>	<code>MazePosition</code> – Startposition des Spielers

Wichtige Methoden:

- `isWalkableForPlayer(int row, int col)` — `true` für FLOOR-Kacheln
- `isWalkableForActiveGhost(int row, int col)` — `true` für FLOOR-Kacheln; GHOST_HOUSE ist für aktive Geister blockiert
- `getDirectionTowardTarget(...)` — Delegiert an `GhostNavigator.directionToTarget()`
- `getDirectionTowardNearestGhostHouseExit(...)` — Delegiert an `GhostNavigator.directionToNearestHouseExit()`
- `getPatrolTargets()` — Liefert vier Patrouillierpunkte nahe den Ecken des Labyrinths
- `getFarthestPatrolTarget(ghostPos, playerPos)` — Liefert den Patrouillierpunkt mit grösstem Manhattan-Abstand zum Spieler

GhostNavigator

Aufgabe: Kapselt alle BFS-basierten Pfadfindungsoperationen für Geister.

`GhostNavigator` ist paketprivat und wird ausschliesslich von `TileMap` instanziiert. Die Klasse arbeitet mit Distanzmatrizen: Ein zweidimensionales `int[][]`-Array wird mit `-1` initialisiert; dann wird vom Zielfeld aus BFS ausgeführt und jede erreichbare Kachel mit ihrer Distanz belegt. Um die beste nächste Richtung zu wählen, werden alle vier Nachbarn des Startfelds verglichen und die Richtung mit dem kleinsten Distanzwert bevorzugt.

Methoden:

Methode	Beschreibung
<code>directionToTarget(startRow, startCol, targetRow, targetCol, currentDir)</code>	BFS vom Ziel aus; wählt Nachbar mit kleinster Distanz; verhindert Umkehrung wenn möglich
<code>directionToNearestHouseExit(startRow, startCol, currentDir)</code>	BFS über Hauskacheln und Ausgangs-Tiles; liefert Richtung zum nächsten Ausgang
<code>hasActivePath(startRow, startCol, targetRow, targetCol)</code>	Prüft Erreichbarkeit ohne Richtungsentscheidung

MazeLoader

Aufgabe: Lädt JSON-Labyrinthdateien und validiert sie vollständig vor der Nutzung.

Validierungsstufen:

1. **Dimensionen:** Exakt 32 Zeilen, jede Zeile exakt 24 Zeichen
2. **Symbole:** Nur erlaubte Zeichen (`#`, `.`, `P`, `G`, `H`, `E`, `o`)
3. **Rahmen:** Alle Randkacheln müssen `#` sein
4. **Spawnzählung:** Genau ein `P`, mindestens vier `G`, mindestens ein `E`
5. **Geisterhaus:** `E` muss an `H` oder `G` angrenzen; jedes `G` muss an `H/G` angrenzen; alle Hauskacheln müssen zusammenhängen
6. **Pfadverbindung:** BFS von der ersten Nicht-Wand-Kachel muss alle Nicht-Wand-Kacheln erreichen
7. **Spielerpfade:** BFS von `P` muss alle spielerzugänglichen Kacheln (kein `#`, `H`, `G`) erreichen

HighscoreManager

Aufgabe: Liest und schreibt `highscores.json` mit Gson; hält stets nur die Top 10 Einträge sortiert nach Punktzahl absteigend.

Fehlerbehandlung: Fehlende Datei gibt leere Liste zurück. Ungültiges JSON oder I/O-Fehler werden abgefangen und geben ebenfalls eine leere Liste zurück. Schreibfehler werden ignoriert – ein Highscore-Problem darf das Spiel nicht unterbrechen.

GamePanel

Aufgabe: Einzige Swing-Komponente mit spielrelevanter Logik. Verarbeitet alle Eingaben und rendert alle Spielzustände.

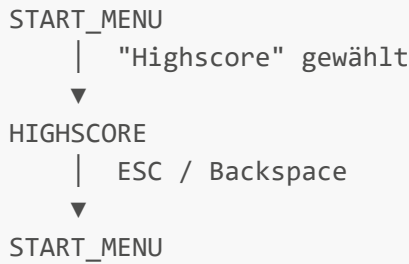
Rendering-Dispatch in `paintComponent(Graphics)`:

```
START_MENU      → renderMainMenu()
NAME_INPUT      → renderNameInput()
MAZE_SELECTION  → renderMazeSelection()
HIGHSCORE       → renderHighscoreScreen()
PLAYING         → renderGameplay()
GAME_OVER / WIN → renderGameplay() + renderEndScreen()
```

Beim Übergang in den Spielzustand (`activateMazeOption`) oder beim Neustart (`activateEndOption`) passt `GamePanel` die Fenstergrösse an das Labyrinth an (`setGameplaySize`). Im Menü beträgt die Fenstergrösse fest 800 × 600 Pixel; im Spiel ergibt sie sich aus Kachelanzahl × Kachelgrösse plus 40 Pixel HUD-Höhe.

6. Spielzustände





Zustandsübergänge werden ausschliesslich durch Methoden auf `World` ausgelöst:

Auslöser	Methode	Zielzustand
„Start“ im Menü	<code>world.showNameInput()</code>	<code>NAME_INPUT</code>
Enter in Namenseingabe	<code>world.confirmPlayerName(name)</code>	<code>MAZE_SELECTION</code>
Labyrinth gewählt	<code>world.startMaze(mazeId)</code>	<code>PLAYING</code>
Alle Pellets eingesammelt	intern in <code>world.update()</code>	<code>WIN</code>
Geisterkollision	intern in <code>world.update()</code>	<code>GAME_OVER</code>
„Restart“	<code>world.restartGame()</code>	<code>PLAYING</code>
„Main Menu“	<code>world.returnToMainMenu()</code>	<code>START_MENU</code>
„Highscore“	<code>world.showHighscores()</code>	<code>HIGHSCORE</code>

7. Spiellogik

Spielstart und Labyrinthinitialisierung

Der Aufruf von `world.startMaze("maze1")` löst `initializeMaze(TileMap)` aus. Diese Methode:

1. Ersetzt `map` durch das neu geladene `TileMap`-Objekt
2. Berechnet `width` und `height` aus Kachelanzahl × Kachelgrösse (24 px)
3. Setzt `score = 0` und `hasSavedHighscore = false`
4. Leert `entities` und `pellets`
5. Erzeugt `Player` an der in der JSON-Datei definierten `P`-Position
6. Erzeugt vier `Ghost`-Objekte an den `G`-Positionen mit zugewiesener Farbe, Persönlichkeit und Verzögerung
7. Erzeugt `Pellet`-Objekte für alle `.`- und `o`-Kacheln

Spielerbewegung

Geschwindigkeit: 200 px/s. Die Bewegungsschleife in `Player.update()` arbeitet mit Delta-Time und fährt in Mikroschritte auf, sodass keine Kachel übersprungen wird. An jedem Kachelmittelpunkt wird die gepufferte Wunschrichtung angewendet, sofern der Zielkachel begehbar ist.

Pellet-System

`World.checkPelletCollection()` prüft jede Frame, ob der AABB des Spielers mit einem noch nicht eingesammelten Pellet überlappt. Ist dies der Fall, wird `pellet.collect()` aufgerufen und `score += 10` ausgeführt.

Gewinn- und Verlustbedingung

- **Sieg:** Nach jeder Pellet-Einsammlung prüft `areAllPelletsCollected()`, ob noch uneingesammelte Pellets vorhanden sind. Ist die Liste leer, wird `endGame(GameState.WIN)` aufgerufen.
- **Niederlage:** `checkGhostCollision()` prüft jede Frame, ob der AABB des Spielers mit einem aktiven Geist überlappt. Ein Geist gilt erst als aktiv (`ghost.isActive()`), wenn er das Geisterhaus vollständig verlassen hat. Kollisionen mit Geistern im Haus beenden das Spiel nicht.

Highscore-Speicherung

Bei `endGame()` wird einmalig (geschützt durch `hasSavedHighscore`) `highscoreManager.addScore(name, score, mazeId, mazeName)` aufgerufen. Der Eintrag wird in die geladene Liste eingefügt, auf 10 Einträge begrenzt und in `highscores.json` persistiert.

8. Ghost-KI

Release-System

Jeder Geist durchläuft drei Phasen, codiert in `GhostReleaseState`:

Phase	Beschreibung
<code>WAITING_IN_HOUSE</code>	Geist steht still; zählt <code>releaseElapsedSeconds</code> hoch
<code>LEAVING_HOUSE</code>	Geist navigiert via BFS zum Ausgangs-Tile
<code>ACTIVE</code>	Geist ist vollständig aktiv und verfolgt den Spieler

Die konfigurierten Verzögerungen in `World`:

Geist	Farbe	Verzögerung
RED	Rot	0 s
PINK	Pink	5 s
BLUE	Cyan	10 s
ORANGE	Orange	15 s

Sobald `releaseElapsedSeconds >= releaseDelaySeconds`, prüft der Geist, ob er noch auf einer Hauskachel steht. Ist dies der Fall, wechselt er in `LEAVING_HOUSE`; andernfalls direkt in `ACTIVE`.

Verlassen des Geisterhauses

In `leaveGhostHouse(World, deltaTime)` wird bei jeder Richtungsentscheidung `TileMap.getDirectionTowardNearestGhostHouseExit()` aufgerufen. Dieser Aufruf delegiert an `GhostNavigator.directionToNearestHouseExit()`, der eine BFS ausschliesslich über Hauskacheln und

Ausgangskacheln (E) ausführt. Der Geist folgt dem kürzesten Pfad zum nächsten E-Tile. Sobald er das E-Tile erreicht und kachelzentriert ist, wird der Zustand auf **ACTIVE** gesetzt.

Zustandsübergänge im aktiven Betrieb

`Ghost.updateState(World)` wird jede Frame aufgerufen. Die Distanz zum Spieler wird mit `Math.hypot()` berechnet:

- Distanz < 20 Pixel → **AttackState**
- Distanz ≥ 20 Pixel → **ChaseState**

AttackState extends **ChaseState** und verhält sich identisch. Die Unterscheidung ist für zukünftige Erweiterungen (z. B. Frightened Mode) vorgesehen.

Pfadfindung im aktiven Betrieb

`GhostNavigator.directionToTarget()` führt eine BFS vom Zielfeld aus durch und befüllt eine Distanzmatrix für alle aktiv begehbaren Kacheln (`isWalkableForActiveGhost`). Aktive Geister können keine Hauskacheln betreten. Aus den vier möglichen Nachbarn des aktuellen Geisterfelds wird der Nachbar mit dem kleinsten Distanzwert gewählt. Eine Umkehrung (180°) wird nur dann erlaubt, wenn keine andere Richtung begehbar ist.

Richtungsentscheidungen werden nur bei kachelzentrierter Position getroffen (`isCenteredOnTile` mit Toleranz 2 Pixel) und nur auf einer neuen Kachel, die sich von der letzten Entscheidungskachel unterscheidet (`isOnNewDecisionTile`).

Ghost-Persönlichkeiten

RED – **RedTargetStrategy**

Gibt direkt die aktuelle Spielerposition als Zielkachel zurück. Keine Zustandsverwaltung, keine Verzögerung.

```
return ghost.getPlayerTile(world);
```

PINK – **PinkTargetStrategy**

Versucht, eine Position bis zu 4 Kacheln vor dem Spieler anzuvisieren. Die Strategie probiert Abstände von 4 bis 1 aus und wählt den ersten, der für aktive Geister begehbar ist und einen gültigen BFS-Pfad vom Geist aus hat. Ist keiner erreichbar, wird die direkte Spielerposition zurückgegeben.

```
Ziel = Spielerposition + (Spielerrichtung × AMBUSH_DISTANCE)
```

BLUE – **BlueTargetStrategy**

Patrouilliert zwischen vier Eckpunkten des Labyrinths (`TileMap.getPatrolTargets()`). Wenn der Spieler innerhalb von 8 Kacheln (Manhattan-Distanz) liegt und der Patrol-Cooldown abgelaufen ist, wechselt Blue für

3 Sekunden in den Verfolgungsmodus. Nach Ablauf der Verfolgungsphase gilt ein Cooldown von 2 Sekunden, bevor erneut verfolgt werden kann. Dadurch entsteht ein gezieltes, aber zeitlich begrenztes Angriffsmuster.

ORANGE – OrangeTargetStrategy

Implementiert eine Hystereselogik zwischen Verfolgen und Rückzug:

- `chasing = false`, Distanz ≥ 10 → wechselt zu `chasing = true`
- `chasing = true`, Distanz ≤ 6 → wechselt zu `chasing = false`

Im Verfolgungsmodus wird die Spielerposition als Ziel zurückgegeben. Im Rückzugsmodus gibt `TileMap.getFarthestPatrolTarget()` den Eckpunkt zurück, der am weitesten vom Spieler entfernt liegt. Die Hystereseschwellen (10 / 6) verhindern schnelles Umschalten bei mittlerer Distanz.

9. Labyrinth-System

JSON-Dateiformat

Jedes Labyrinth ist eine JSON-Ressource unter `/mazes/maze{n}.json`:

```
{
  "id": "maze1",
  "name": "Maze 1",
  "pattern": [
    "#####",
    "#.....#.....#.....#.....#",
    "#.###.#.###.#.###.#.###.#",
    "...",
    "#.....##E##.....#",
    "#.#####.##HGHG.#####.#",
    "#.#.....##HHHH.....#.#",
    "#.#.#####.GHGH.#####.#.#",
    "...",
    "#####",
  ]
}
```

Unterstützte Symbole

Symbol	Bedeutung
#	Wand
.	Boden mit Pellet
	Boden ohne Pellet
P	Spieler-Spawn
G	Geister-Spawn (Hauskachel)

Symbol	Bedeutung
H	Geisterhausboden (kein Spawn)
E	Geisterhausausgang
o	Boden mit Pellet (reserviert für Power-Pellet)

Abmessungen

Alle Labyrinth haben exakt 24 Spalten und 32 Zeilen. Jede Kachel misst 24×24 Pixel. Das ergibt eine Spielfeldgröße von 576×768 Pixel.

Spawnpunkte und Geisterhaus

Die vier G-Tiles werden in der Reihenfolge ihres Vorkommens (zeilenweise, von oben) als Spawnpunkte der Geister gelesen. Der erste G-Tile gehört RED (0 s), der zweite PINK (5 s), der dritte BLUE (10 s) und der vierte ORANGE (15 s).

Das Geisterhaus besteht aus zusammenhängenden G- und H-Kacheln. Der Ausgang ist durch mindestens ein E-Tile markiert, das an eine Hauskachel angrenzt.

Validierungsablauf in MazeLoader

```

load(resourcePath)
├── Gson.fromJson() → MazeData
└── validate(MazeData)
    ├── validateDimensions()
    ├── validateSymbolsAndCountSpawns()
    ├── validateRequiredSpawns()
    ├── validateGhostHouse()
    │   ├── E muss an H/G angrenzen
    │   ├── G muss an H/G angrenzen
    │   └── validateSingleGhostHouse() via BFS
    ├── validateConnectedPaths() via BFS (alle Nicht-Wand-Kacheln)
    └── validatePlayerPaths() via BFS (alle spielerzugänglichen
Kacheln)

```

10. Highscore-System

Dateistruktur

highscores.json liegt im Arbeitsverzeichnis des Prozesses. Format:

```

[
  {
    "name": "Alice",

```

```
"score": 2340,
"mazeId": "maze3",
"mazeName": "Maze 3"
},
{
"name": "Bob",
"score": 1870,
"mazeId": "maze1",
"mazeName": "Maze 1"
}
]
```

Verwaltungslogik

`prepareHighscores(List<HighscoreEntry>)` wird bei jedem Lese- und Schreibvorgang aufgerufen:

1. Filtert `null`-Einträge und Einträge ohne Namen
2. Sortiert absteigend nach `score`
3. Begrenzt auf maximal 10 Einträge via `Stream.limit(10)`

Fehlerbehandlung

Alle Methoden von `HighscoreManager` fangen `IOException`, `JsonParseException` und `SecurityException` ab. Ein defekter Highscore beeinflusst den Spielablauf nicht.

11. Tests

Das Projekt enthält 72 JUnit-Jupiter-Testmethoden in sechs Testklassen.

Testklasse	Anzahl Tests	Getestete Aspekte
<code>GhostTest</code>	18	Verzögerungslogik, Hausaustritt, Bewegung auf Kachelgitter, Persönlichkeitszuweisung, Richtungsentscheidungen, Reset
<code>WorldTest</code>	14	Spielzustände, Pellet-Einsammeln, Geisterkollision, Neustart, Highscore-Speicherung
<code>PlayerTest</code>	10	Bewegung ohne Richtung, Delta-Time-Bewegung, Warten auf Kachelmitte, Lane-Zentrierung
<code>MazeLoaderTest</code>	16	Alle fünf Labyrinth, Dimensionen, Spawnzählung, Validierungsfehler, Pfadverbindung, Sackgassenzählung
<code>HighscoreManagerTest</code>	8	Fehlende Datei, ungültiges JSON, Top-10-Sortierung, Labyrinthzuordnung
<code>GamePanelTest</code>	6	Fenstergrößen, Entity-Größen, Tastatureingaben, Mausinteraktion

Die Spiellogiktests in `GhostTest`, `WorldTest` und `PlayerTest` verwenden kein Swing. `World` und alle Entitäten werden direkt instanziiert und mit einer `TileMap` versehen. Dadurch laufen diese Tests schnell und ohne grafische Umgebung.

12. Herausforderungen und Lösungen

Geisterbewegung ohne Oszillation

Problem: Eine naive BFS-Implementierung, die bei jeder Frame-Aktualisierung neu ausgeführt wird, führte dazu, dass Geister an Kreuzungen ständig die Richtung wechselten. Das Ergebnis war ein unnatürliches Hin- und-Her-Bewegungsmuster.

Lösung: Richtungsentscheidungen werden ausschliesslich getroffen, wenn der Geist (a) kachelzentriert ist (Toleranz 2 px) und (b) sich auf einer neuen Kachel befindet, die von der letzten Entscheidungskachel abweicht (`lastDecisionRow`, `lastDecisionCol`). In `chooseIdleDirection` wird die aktuelle Richtung bevorzugt, wenn sie weiterhin begehbar ist. Die BFS in `GhostNavigator.avoidReverse()` unterdrückt zusätzlich die 180°-Umkehrung, solange andere Optionen verfügbar sind.

Ergebnis: Geister wirken zielgerichtet und ändern ihre Richtung nur an echten Kreuzungen oder Sackgassen.

Geisterhausaustritt ohne Hängenbleiben

Problem: Das Geisterhaus ist durch Wände vom Rest des Labyrinths abgetrennt. Die allgemeine BFS für aktive Geister behandelt Hauskacheln als blockiert und kann daher nicht für den Hausaustritt verwendet werden.

Lösung: `GhostNavigator.directionToNearestHouseExit()` verwendet eine separate BFS, die ausschliesslich über Hauskacheln (H, G) und Ausgangskacheln (E) läuft (`isHouseReleaseTile`). Das Ziel ist jedes E-Tile. Geister folgen diesem Pfad bis zur Kachelmitte des E-Tiles und wechseln dann in `ACTIVE`.

Ergebnis: Geister verlassen das Haus zuverlässig von jeder Startposition, unabhängig von der Labyrinthgeometrie.

Blockierung aktiver Geister am Geisterhaus

Problem: Die ursprüngliche Implementierung von `isWalkableForActiveGhost()` delegierte an `isWalkableForPlayer()`, was Hauskacheln als begehbar behandelte. Aktive Geister konnten rechnerisch wieder ins Haus navigiert werden.

Lösung: `isWalkableForActiveGhost()` schliesst explizit `GHOST_HOUSE`-Kacheln aus. Zusätzlich prüft `World.isBlockedFor(Entity, row, col)`: Ist der Entitätstyp `Ghost` und `ghost.isActive()`, wird die Hauskachel als blockiert zurückgegeben.

Ergebnis: Ein einmal aktiver Geist kann das Geisterhaus nicht mehr betreten.

Kachelbasierte Bewegung mit flüssiger Animation

Problem: Reine Kachelsprünge (ganzzahlige Positionssprünge) wirken abgehackt. Eine rein pixelbasierte Bewegung erschwert jedoch die Kollisionserkennung und Pfadfindung.

Lösung: Entitäten bewegen sich kontinuierlich in Pixelkoordinaten (`double x, y`), werden aber bei Richtungsentscheidungen auf Kachelmittelpunkte eingerastet. Die `centerOnLane()`-Methode in `MovingEntity` korrigiert pro Frame die Querabweichung von der Kachelmitte mit halber Geschwindigkeit, sodass der Drift aus dem Korridor sanft und ohne visuelles Zittern beseitigt wird.

Ergebnis: Flüssige Animation bei gleichzeitig sauberer Gitterlogik.

Mehrfache BFS-Validierung im MazeLoader

Problem: Es ist möglich, ein Labyrinth zu definieren, das syntaktisch korrekt ist, aber logisch ungültig – z. B. mit einem unerreichbaren Korridor, einem vom Geisterhaus getrennten `G`-Tile oder einem Spieler-Spawn, von dem nicht alle Pellets erreichbar sind.

Lösung: `MazeLoader.validate()` führt drei separate BFS-Durchläufe durch: (1) Verbindungsprüfung aller Nicht-Wand-Kacheln, (2) Verbindungsprüfung des Geisterhauses, (3) Erreichbarkeitsprüfung aller spielerzugänglichen Kacheln vom `P`-Tile aus. Jeder Fehler wirft eine `IllegalArgumentException` mit präziser Fehlermeldung.

Ergebnis: Fehlerhafte Labyrinth werden beim Laden abgefangen, bevor sie zur Laufzeit zu undefiniertem Verhalten führen.

13. Reflexion

Was gut funktioniert hat

Die Entscheidung, `GhostTargetStrategy` als Interface einzuführen, hat sich als eine der wertvollsten Architekturentscheidungen des Projekts erwiesen. Jede Persönlichkeit liess sich vollständig isoliert entwickeln und testen, ohne andere Teile des Systems zu berühren. Nachträgliche Anpassungen an einzelnen Persönlichkeiten – zum Beispiel das Einführen der Hysterese bei Orange oder das Fallback-Verhalten bei Pink – waren in wenigen Zeilen umzusetzen.

Die strikte Trennung von Spiellogik und Rendering hat sich ebenfalls bewährt. Die Tests für `Ghost`, `Player` und `World` laufen komplett ohne Swing und sind dadurch schnell, deterministisch und einfach zu schreiben.

Die BFS-basierte Pfadfindung in `GhostNavigator` ist zuverlässig und produziert konsistente Ergebnisse. Die Entscheidung, die Distanzmatrix vom Ziel ausgehend zu berechnen (statt vom Start), ermöglicht es, mit einem einzigen BFS-Durchlauf alle Nachbarrichtungen zu vergleichen.

Was man heute anders machen würde

`AttackState` ist derzeit identisch mit `ChaseState` – er wurde als separate Klasse eingeführt in Erwartung einer Frightened-Mode-Erweiterung, die nicht umgesetzt wurde. Das erzeugt eine leere Abstraktion. Eine sauberere Lösung wäre gewesen, den Nahkampfbereich direkt in `ChaseState` zu integrieren oder `AttackState` erst dann einzuführen, wenn er sich tatsächlich vom `ChaseState` unterscheidet.

Die **TileMap**-Klasse hat im Laufe der Entwicklung mehrere Verantwortlichkeiten aufgenommen: Sie verwaltet das Kachelgitter, hält Spawnpunkte, berechnet Pixelkoordinaten und delegiert Pfadfindungsanfragen. Eine stärkere Aufteilung – z. B. eine separate **SpawnRegistry** oder ein eigenständiges **PathfindingFacade** – würde die Klasse kohärenter machen.

Der Legacy-Kartenpfad (**loadLegacyMap**) in **TileMap** unterstützt das alte Textformat (**/maps/map.txt**) für Tests. Dieser Code ist heute nur noch für Tests relevant und könnte in eine Testhelfer-Klasse ausgelagert werden.

Sinnvolle Erweiterungen

Power-Pellets: Das Symbol **o** ist im Loader bereits reserviert. Die Umsetzung erfordert einen vierten Ghost-Zustand (**FrightenedState**), einen Timer sowie eine Umkehrung der Kollisionslogik für die Dauer der Schreckphase.

Lebens-System: Würde minimale Änderungen an **World** erfordern: ein **lives**-Attribut, eine Unterscheidung zwischen „Runde verloren“ und „Spiel verloren“ sowie eine Respawn-Verzögerung ohne vollständigen Reset des Pellet-Zustands.

Schwierigkeitseinstellungen: Da Geschwindigkeiten als Konstanten in **Player** (**200.0**) und **Ghost** (**100.0**) sowie Freigabeverzögerungen in **World** als **List<Double>** definiert sind, liessen sich Schwierigkeitsstufen durch Parametrisierung dieser Werte mit minimalem Aufwand einführen.
